STATIC ANALYSIS WITH CEDAR TOOLS

Running tests on your policies shows how they perform in specific scenarios. But what about questions like these: Are there any unreachable policies? Do two policies conflict with each other? Is there a case where no policy applies at all?

The Cedar Analysis project (https://aws.amazon.com/blogs/opensource/introducing-cedar-analysis-open-source-tools-for-verifying-authorization-policies/), made open source by AWS, offers tools for the static verification of policy sets. Rather than run policies with sample requests, Cedar Analysis examines the policies themselves, testing all possible inputs against the schema. It can identify issues such as:

Conflicts—like when one policy grants access but another denies it under the same conditions

Redundancy—multiple overlapping policies that seldom take effect

Gaps—actions or resources not addressed by any policy, which can lead to unexpected implicit denials

Static analysis enhances testing. At ACME, engineers still run a suite of request-driven tests, but they also perform Cedar Analysis within CI pipelines to identify structural issues before policies are deployed. This approach provides greater confidence that the policy set is both sound and complete, minimizing surprises during runtime.

For teams adopting Cedar, integrating testing and analysis provides a more robust safety net than depending on either method alone.

Testing policies effectively is essential; it transforms Cedar from just a policy language into a dependable record-keeping system for authorization. By combining request-driven tests with static analysis, ACME guarantees that its policies not only are syntactically correct but also enforce the appropriate rules, address real-world scenarios, and stay resilient as the